

COMPILATION

Cours de préparation — S7 M1 Informatique (ILIA)

De la théorie des langages formels à la génération de code : automates, grammaires, analyse lexicale, syntaxique, sémantique et génération de code — avec exercices corrigés.

Sommaire

1. Introduction : à quoi sert un compilateur ?
2. Langages formels, grammaires et automates
3. Analyse lexicale
4. Analyse syntaxique
5. Analyse sémantique
6. Génération de code
7. Exercices récapitulatifs

Chapitre 1 — À quoi sert un compilateur ?

Un **compilateur** est un programme qui traduit un texte écrit dans un langage source (par exemple du C, du Java ou un langage inventé pour un TP) vers un langage cible : du code assembleur, du bytecode, ou même un autre langage de haut niveau. Cette traduction se fait en plusieurs étapes indépendantes, formant ce qu'on appelle le **pipeline de compilation**.

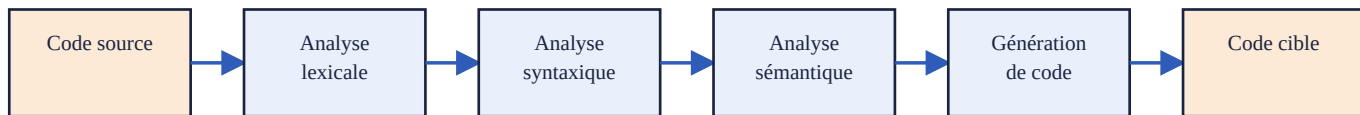


Figure 1.1 — Le pipeline classique d'un compilateur

Chaque étape a un rôle précis et communique avec la suivante via une représentation intermédiaire (suite de tokens, arbre syntaxique, arbre annoté, code intermédiaire). C'est cette découpe modulaire qui rend un compilateur maintenable et qui justifie qu'on étudie chaque étape séparément.

À retenir : compiler, ce n'est pas « traduire d'un coup ». C'est une chaîne de transformations, chacune vérifiant et enrichissant la représentation du programme.

Pourquoi une matière théorique ?

- Elle relie directement à ta matière d'**algorithmique avancée** (automates, grammaires, complexité).
- Elle explique *comment* fonctionnent les langages que tu utilises déjà tous les jours.
- Les concepts (automates, arbres, parseurs) reviennent constamment ailleurs : regex, JSON parsing, interprètes, moteurs de règles, analyseurs syntaxiques pour données géographiques (utile pour ton parcours ILIA !).

Chapitre 2 — Langages formels, grammaires et automates

2.1 Alphabets, mots, langages

Un **alphabet** Σ est un ensemble fini de symboles. Un **mot** est une suite finie de symboles de Σ (le mot vide se note ϵ). Un **langage** L est un sous-ensemble de Σ^* , l'ensemble de tous les mots possibles sur Σ .

Exemple : $\Sigma = \{a, b\}$. Le langage $L = \{ \text{mots contenant un nombre pair de } a \}$ est un sous-ensemble de Σ^* . On veut souvent décrire un langage de façon finie, même s'il contient une infinité de mots — c'est le rôle des grammaires et des automates.

2.2 Expressions régulières

Une **expression régulière** décrit un langage à l'aide d'opérateurs simples : concaténation, union ($|$), étoile de Kleene ($*$, répétition 0 ou plusieurs fois).

Expression	Langage décrit
a^*	$\epsilon, a, aa, aaa, \dots$
$(a b)^*$	tous les mots sur $\{a,b\}$, y compris ϵ
$a(a b)^*abb$	mots commençant par a et finissant par abb

2.3 Automates finis

Un **automate fini** (AFD/AFN) est un dispositif abstrait à états qui reconnaît un langage régulier. Il est défini par : un ensemble d'états Q , un alphabet Σ , une fonction de transition δ , un état initial q_0 , et un ensemble d'états finaux F .

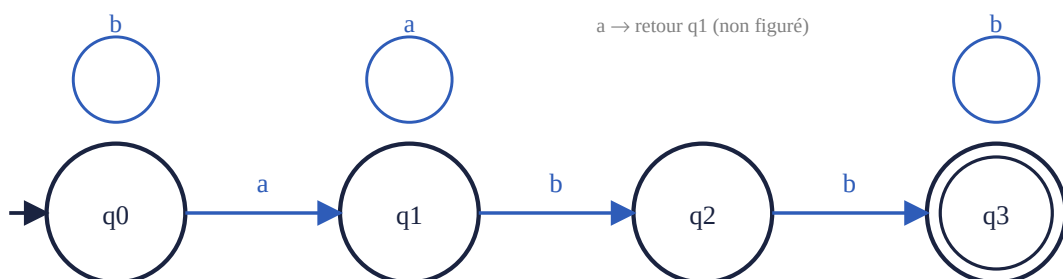


Figure 2.1 — Automate reconnaissant le langage $a(a|b)^*abb$ (structure simplifiée)

Tout langage décrit par une expression régulière peut être reconnu par un automate fini, et réciproquement (théorème de Kleene). C'est ce lien qui est exploité par l'analyseur lexical.

2.4 Grammaires hors-contexte

Les langages de programmation ne sont en général **pas réguliers** (il faut savoir compter des parenthèses imbriquées, par exemple). On utilise alors des **grammaires hors-contexte** (context-free grammars), définies par un ensemble de règles de production.

Exemple : grammaire d'expressions arithmétiques

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

Cette grammaire est **ambiguë** (une même chaîne peut avoir plusieurs arbres de dérivation), ce qui pose problème pour un compilateur : on la réécrit en général pour fixer la priorité des opérateurs et l'associativité.

Exercice 1 — Automates et expressions régulières

- a) Donnez une expression régulière décrivant les identifiants classiques d'un langage de programmation (une lettre suivie de lettres ou chiffres).
- b) Dessinez (sur papier) un automate fini reconnaissant le langage $(ab)^*$ sur $\Sigma = \{a,b\}$.
- c) Le langage $\{ a^n b^n, n \geq 0 \}$ est-il régulier ? Justifiez brièvement.

Correction

a) $[a-zA-Z][a-zA-Z0-9]^*$

b) Un automate à 3 états : q_0 (initial), q_1 , q_2 (final = q_0 lui-même si on accepte le mot vide). Transition $q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_0$.

c) Non : ce langage nécessite de « compter » un nombre non borné de a avant de vérifier l'égalité avec le nombre de b , ce qu'un automate fini (mémoire bornée) ne peut pas faire. C'est un exemple classique de langage hors-contexte non régulier (démontrable par le lemme de pompage).

Chapitre 3 — Analyse lexicale

L'**analyseur lexical** (ou *lexer/scanner*) lit le code source caractère par caractère et le regroupe en unités porteuses de sens appelées **tokens** (ou lexèmes). Il élimine au passage les espaces, tabulations et commentaires.

Exemple : lexer sur une ligne de code

```
total = prix * 3 + 1
```

Lexème	Type de token
total	IDENTIFIANT
=	AFFECTATION
prix	IDENTIFIANT
*	OPERATEUR_MUL
3	ENTIER
+	OPERATEUR_ADD
1	ENTIER

Concrètement, chaque catégorie de token (identifiant, entier, opérateur...) est décrite par une expression régulière ; ces expressions sont converties en automate, souvent via un outil générateur de lexer (lex/flex, ou une implémentation manuelle).

Point de vigilance classique : la règle du *plus long match* (maximal munch) — le lexer choisit toujours le plus long lexème valide possible. Exemple : **foobar** est lu comme un seul identifiant, pas comme **foo** puis **bar**.

Exercice 2 — Découpage en tokens

Découpez la ligne suivante en tokens et donnez le type de chacun :

```
if (x >= 10) y = x + 1;
```

Correction

```
if → MOT_CLE | ( → PARENTHESE_OUVRANTE | x → IDENTIFIANT | >= →  
OPERATEUR_COMPARAIISON | 10 → ENTIER | ) → PARENTHESE_FERMANTE | y →  
IDENTIFIANT | = → AFFECTATION | x → IDENTIFIANT | + → OPERATEUR_ADD | 1 → ENTIER | ;  
→ POINT_VIRGULE
```

Chapitre 4 — Analyse syntaxique

L'**analyseur syntaxique** (ou *parser*) prend la suite de tokens produite par le lexer et vérifie qu'elle respecte la grammaire du langage. Si oui, il construit un **arbre syntaxique** (parse tree ou AST) représentant la structure du programme.

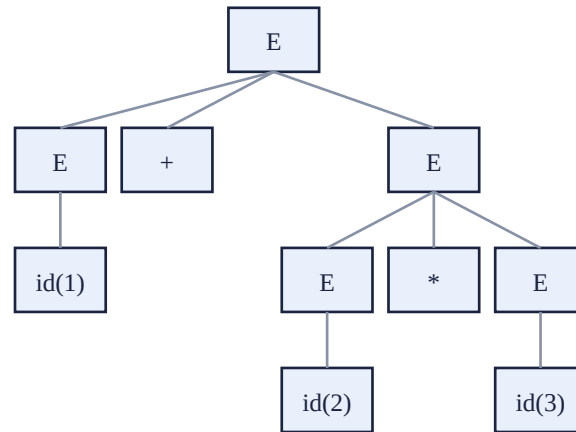


Figure 4.1 — Arbre syntaxique de $id + id * id$ avec $E \rightarrow E+E \mid E * E \mid id$

4.1 Deux grandes familles de parseurs

Type	Principe	Exemple d'usage
Descendant (LL)	Construit l'arbre depuis la racine, prédit la règle à appliquer en regardant le prochain token	Parseurs descendants, faciles à écrire à la main
Ascendant (LR)	Construit l'arbre depuis les feuilles, empile puis réduit (shift-reduce)	Générateurs de parseurs comme yacc/bison, plus puissants mais moins intuitifs

4.2 Grammaire non ambiguë avec priorités

Pour éliminer l'ambiguïté de $E \rightarrow E+E \mid E * E \mid (E) \mid id$, on stratifie la grammaire par niveau de priorité :

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

Cette réécriture force le $*$ à se lier plus fortement que le $+$, et impose l'associativité à gauche — exactement ce qu'on attend d'une expression arithmétique classique.

Exercice 3 — Construire un arbre syntaxique

En utilisant la grammaire non ambiguë ci-dessus ($E \rightarrow E+T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid id$), dessinez (sur papier) l'arbre syntaxique de : $id + id * id$. Combien de fois la règle $T \rightarrow F$ est-elle utilisée ?

Correction

L'arbre a pour racine $E \rightarrow E + T$. La branche gauche $E \rightarrow T \rightarrow F \rightarrow \text{id}$ donne le premier id. La branche droite $T \rightarrow T * F$ se développe en $T \rightarrow F \rightarrow \text{id}$ (deuxième id) et $F \rightarrow \text{id}$ (troisième id). La règle $T \rightarrow F$ est utilisée deux fois : une fois pour réduire le premier id, une fois pour le dernier facteur multiplié. Cette construction respecte bien la priorité du $*$ sur le $+$.

Chapitre 5 — Analyse sémantique

Une fois l'arbre syntaxique construit, il reste à vérifier que le programme a un **sens** : les variables sont-elles déclarées avant usage ? Les types sont-ils compatibles ? C'est le rôle de l'**analyse sémantique**, qui s'appuie sur une **table des symboles**.

Table des symboles : structure associant à chaque identifiant ses informations (type, portée, adresse mémoire...). Elle est consultée et mise à jour tout au long du parcours de l'arbre syntaxique.

Exemples d'erreurs détectées à cette étape

- Utilisation d'une variable non déclarée.
- Incompatibilité de types : "texte" + 3 selon le langage.
- Mauvais nombre ou type d'arguments lors d'un appel de fonction.
- Redéfinition d'un identifiant dans la même portée.

On distingue bien cette étape des erreurs *syntactiques* (structure du texte incorrecte) détectées plus tôt : une phrase peut être syntaxiquement correcte mais sémantiquement absurde, par exemple `int x = "bonjour";` en C.

Exercice 4 — Erreurs lexicales, syntaxique ou sémantique ?

Pour chacun des cas suivants, indiquez à quelle étape l'erreur serait détectée :

- a) `int x = 5` (point-virgule oublié)
- b) `int x = 5 @ 2;` (symbole @ inconnu)
- c) `int x = "hello";`
- d) `y = y + 1;` où y n'a jamais été déclaré

Correction

a) Syntaxique (structure de l'instruction invalide). b) Lexicale (@ n'appartient à aucune catégorie de token valide). c) Sémantique (structure correcte, mais type incompatible). d) Sémantique (variable absente de la table des symboles).

Chapitre 6 — Génération de code

La dernière étape traduit l'arbre syntaxique annoté vers un **code cible** : assembleur, bytecode (Java, Python...), ou un langage intermédiaire (souvent une forme proche de l'assembleur à trois adresses, dite *three-address code*).

Exemple : code à trois adresses

Instruction source : `total = prix * 3 + 1`

```
t1 = prix * 3
t2 = t1 + 1
total = t2
```

Chaque instruction intermédiaire ne comporte qu'un seul opérateur, ce qui facilite les étapes suivantes : allocation de registres, optimisations (élimination de code mort, propagation de constantes...), puis traduction finale vers le vrai langage cible.

À retenir : plus la représentation intermédiaire est simple et régulière, plus il est facile d'écrire des optimisations génériques, indépendantes du langage cible final.

Exercice 5 — Génération de code à trois adresses

Traduisez l'instruction suivante en code à trois adresses :

`resultat = (a + b) * (c - d);`

Correction

```
t1 = a + b
t2 = c - d
t3 = t1 * t2
resultat = t3
```

Chapitre 7 — Exercices récapitulatifs

Exercice 6 — Parcours complet du pipeline

Soit l'instruction : $z = x * y + 2;$

- a) Listez les tokens produits par l'analyseur lexical.
- b) Donnez une grammaire simple (type $E \rightarrow E+T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{id} \mid \text{num}$) permettant de construire l'arbre syntaxique de l'expression $x * y + 2$.
- c) Quelles vérifications sémantiques seraient nécessaires avant de générer du code ?
- d) Donnez le code à trois adresses correspondant.

Correction

a) z (IDENTIFIANT), $=$ (AFFECTATION), x (IDENTIFIANT), $*$ (OPERATEUR_MUL), y (IDENTIFIANT), $+$ (OPERATEUR_ADD), 2 (ENTIER), $;$ (POINT_VIRGULE)

b) Grammaire : $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{id} \mid \text{num}$. L'arbre a pour racine $E \rightarrow E + T$, avec la branche gauche réduite à $T \rightarrow F \rightarrow \text{id}$ (x), et le terme $x*y$ construit via $T \rightarrow T * F$.

c) Vérifier que x , y et z sont déclarés, que leurs types sont compatibles entre eux et avec l'opération demandée (multiplication puis addition), et que z peut recevoir le résultat (types compatibles pour l'affectation).

d) $t1 = x * y$

$t2 = t1 + 2$

$z = t2$

Exercice 7 — Question ouverte — synthèse

Expliquez en quelques phrases pourquoi on sépare l'analyse syntaxique (est-ce que la phrase est bien formée ?) de l'analyse sémantique (est-ce que la phrase a un sens ?), plutôt que de tout vérifier en une seule passe.

Correction

Séparer les deux permet de garder chaque étape simple et réutilisable : la grammaire hors-contexte décrit uniquement la forme du programme, indépendamment de son sens, ce qu'un parseur générique sait vérifier efficacement. Les règles sémantiques (typage, portée) dépendent souvent du contexte global du programme et ne peuvent pas toujours être exprimées dans une grammaire hors-contexte. Cette séparation facilite aussi la détection d'erreurs : on sait immédiatement si une erreur vient de la structure du texte ou de son sens.